Plano de Projeto: P2P com Balanceamento de Carga Dinâmico

Visão Geral do Projeto

O objetivo deste projeto é desenvolver um sistema distribuído autônomo que demonstre o conceito de balanceamento de carga horizontal através de uma arquitetura P2P. Cada nó "Master" gerencia seu próprio conjunto de nós "Worker" (uma *Farm*). Os Masters monitoram sua carga de trabalho e, ao atingir um limiar de saturação, negociam dinamicamente o empréstimo de Workers de um Master vizinho para lidar com o excesso de requisições. A comunicação e a negociação entre os Masters devem seguir um protocolo de consenso, garantindo que a transferência de recursos seja coordenada e acordada.

O principal desafio é garantir a autonomia entre os sistemas desenvolvidos por diferentes equipes, permitindo que eles se interconectam e colaborem sem conhecimento prévio da implementação interna um do outro, apenas do protocolo de comunicação.

Objetivos

- **O1. Implementar a Arquitetura P2P:** Criar um nó Master capaz de gerenciar (iniciar, parar, monitorar) um conjunto de Workers (Farm).
- **O2. Simular Carga de Trabalho:** Desenvolver um mecanismo para simular requisições chegando a um Master, permitindo o monitoramento da carga.
- **O3. Implementar o Monitoramento de Saturação:** O Master deve ser capaz de identificar quando o número de requisições excede um limiar (threshold) pré-definido.
- **O4. Desenvolver o Protocolo de Conversa Consensual:** Projetar e implementar um protocolo robusto para que um Master saturado possa solicitar ajuda, um Master vizinho possa responder, e ambos possam coordenar o redirecionamento de Workers.
- **O5. Implementar o Redirecionamento Dinâmico de Workers:** Um Master vizinho deve ser capaz de instruir um ou mais de seus Workers a se reportarem temporariamente ao Master saturado.
- **O6. Garantir a Autonomia e Interoperabilidade:** O sistema deve funcionar em conjunto com o sistema de outra equipe, comunicando-se exclusivamente através do protocolo definido.

3. Arquitetura Proposta

O sistema será composto por três entidades principais:

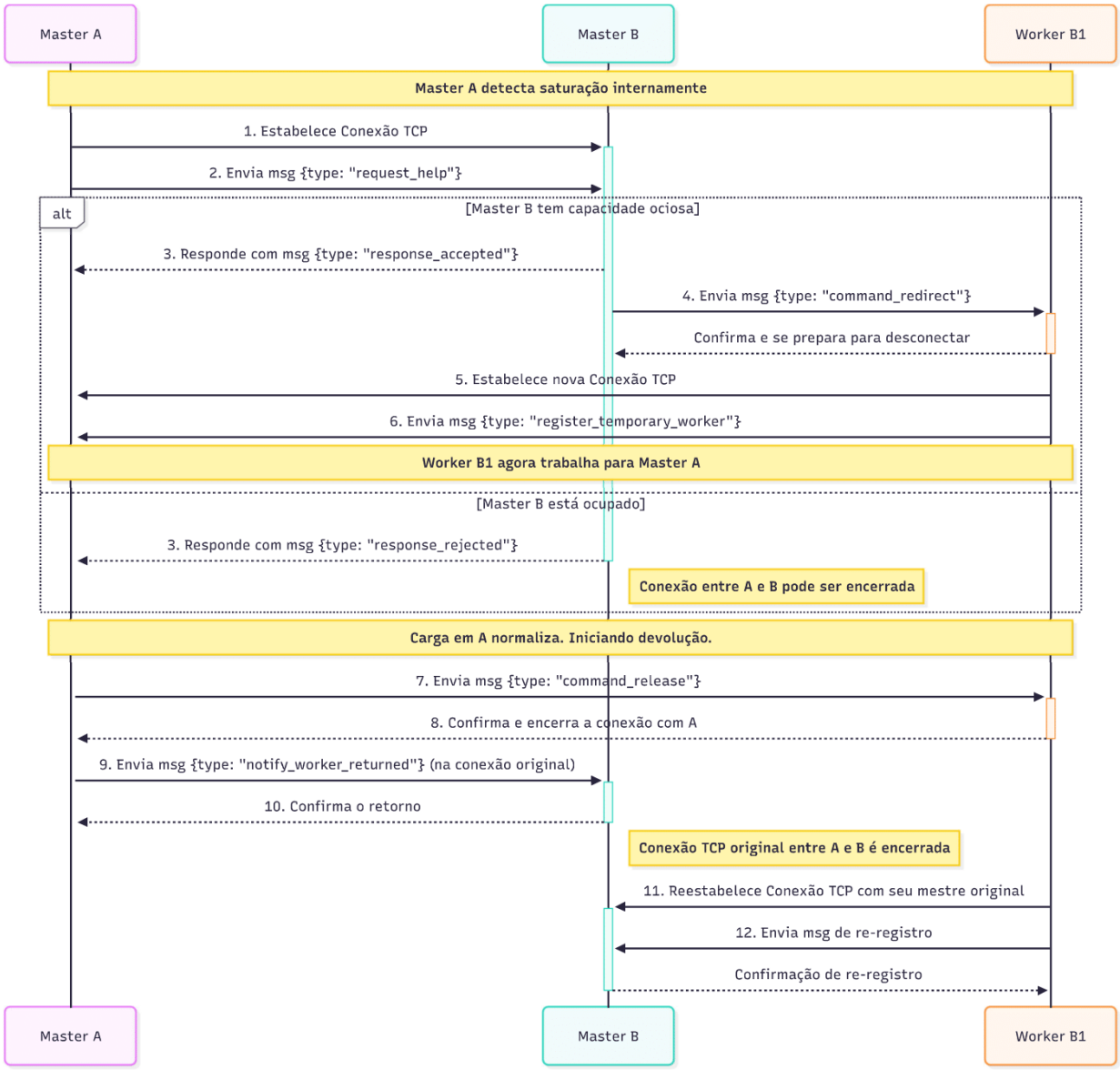
1. **Nó Master:**
   - Recebe requisições de clientes (simuladas).
   - Mantém uma lista de Workers em sua Farm.
   - Distribui requisições entre seus Workers.
   - Monitora constantemente o número de requisições pendentes.
   - Inicia o "Protocolo de Conversa Consensual" quando requisições > threshold.
   - Gerencia Workers "emprestados" de outros Masters.
2. **Nó Worker:**
   - Recebe uma tarefa do seu Master atual e a processa (ex: um cálculo simples, um sleep para

simular trabalho).

- Deve ser capaz de se desconectar do seu Master original e se conectar a um novo Master quando instruído.

3. **Protocolo de Comunicação (Mensageria):**
   - A espinha dorsal do projeto. Define as regras para a negociação.
   - Será baseado em trocas de mensagens, por exemplo, via API REST, gRPC ou Sockets customizados com formato JSON.

Exemplo Diagrama da Comunicação Master <-> Master (Saturação):



Considerações de Implementação

- **Message Delimiter:** Em um stream de TCP, você precisa de uma forma de saber onde uma mensagem JSON termina e a próxima começa. A abordagem mais simples é enviar um caractere de nova linha (\n) após cada objeto JSON. O receptor lê o socket até encontrar um \n, e então processa o que recebeu como um JSON completo.
- **Escuta Contínua:** Cada Master e Worker deve rodar em um loop infinito, escutando por novas conexões (se for servidor) ou novas mensagens em conexões existentes.
- **Threads/AsyncIO:** É fundamental usar concorrência (threads ou programação assíncrona) para que um Master possa se comunicar com múltiplos outros Masters e Workers simultaneamente sem bloquear seu processo principal.
- **Protocolos de Comunicação devem seguir o padrão estabelecido em sala.**

PAYLOAD OFICIAL:

|1| HEARTBEAT Worker A1 - Servidor A - Verificação de Atividade.

|1.1| Worker A1 → Servidor A - Worker A1 pergunta ao Servidor A se ele está ativo.

PAYLOAD:

```
{
    "SERVER_UUID": "...",
    "TASK": "HEARTBEAT"
}
```

|1.2| Servidor A → Worker A1 - Servidor A responde que está ativo.

PAYLOAD:

```
{
    "SERVER_UUID": "...",
    "TASK": "HEARTBEAT",
    "RESPONSE": "ALIVE"
}
```